

actividad3

February 27, 2025

1 Actividad Segmentación de Imágenes

Nombre: Manuel Ramírez Galván

Numero de Actividad: 3

Fecha: 26/02/2025

Objetivos: Desarrollar e implementar técnicas de segmentación de imágenes basadas en detección de bordes, contornos y regiones, utilizando operadores diferenciales, umbralización y algoritmos de crecimiento de regiones.

1.1 Ejercicio 1: Importar bibliotecas necesarias

```
[1]: from google.colab import drive
drive.mount('/content/drive')
import cv2
import matplotlib.pyplot as plt
import numpy as np
```

Mounted at /content/drive

1.2 Ejercicio 2: Detección de Bordes con el Filtro Sobel

Objetivo: Aplicar el filtro Sobel para detectar bordes en una imagen mediante gradientes.

Pasos: 1. Cargar la imagen herramientas con `cv2.imread()`. 2. Divide la imagen en canales usando `cv2.split()` 3. Aplicar el operador Sobel en las direcciones X e Y, usando `cv2.Sobel()` 4. Calcula la magnitud del gradiente usando `np.sqrt()` 5. Visualizar la imagen original y la imagen con los bordes detectados usando `plt.subplot()`.

```
[26]: img1 = cv2.imread("/content/drive/MyDrive/Colab Notebooks/
↳2_3_Segmentacion_de_Imagenes/data/herramientas.jpg", cv2.IMREAD_GRAYSCALE)

channels = cv2.split(img1)
sobel_channels = []

for channel in channels:
    sobel_x = cv2.Sobel(channel, cv2.CV_64F, 1, 0, ksize=3)
    sobel_y = cv2.Sobel(channel, cv2.CV_64F, 0, 1, ksize=3)
```

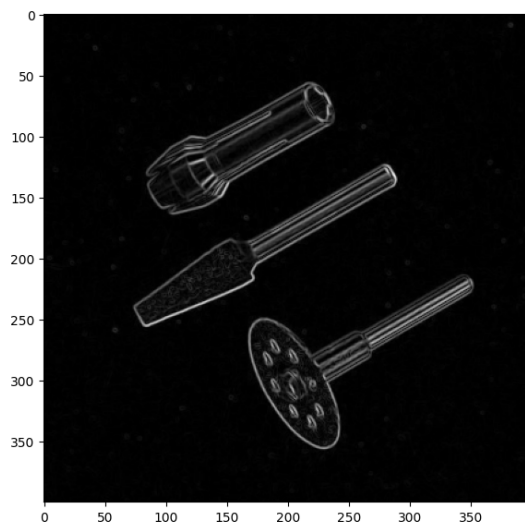
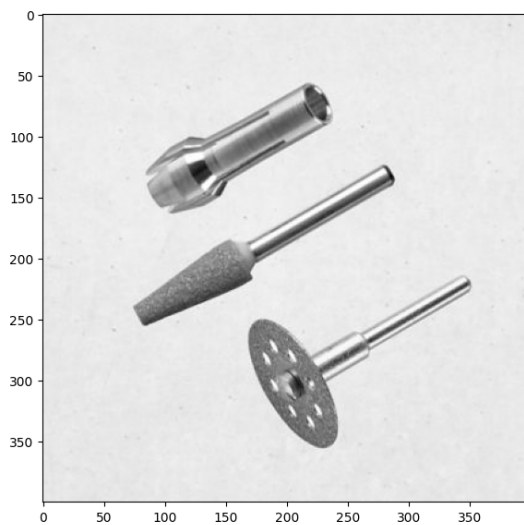
```

sobel_magnitude = np.sqrt(sobel_x**2 + sobel_y**2)
sobel_channels.append(np.uint8(255 * sobel_magnitude / np.
↪max(sobel_magnitude)))

sobel_combined = cv2.merge(sobel_channels)

plt.figure(figsize=(15,10))
plt.subplot(1, 2, 1)
plt.imshow(img1, cmap='gray')
plt.subplot(1, 2, 2)
plt.imshow(sobel_combined, cmap='gray')
plt.show()

```



1.3 Ejercicio 3: Detección de Bordes con Canny

Objetivo: Aplicar el detector de Canny en cada canal de una imagen a color y fusionar los resultados. **Pasos:** 1. Divide la imagen en canales usando `cv2.split()` 2. Aplicar el filtro Canny, usando `cv2.Canny()` 3. Realiza la union de los canales usando `cv2.merge()` 4. Visualizar la imagen original y la imagen con los bordes detectados usando `plt.subplot()`.

```

[27]: channels = cv2.split(img1)

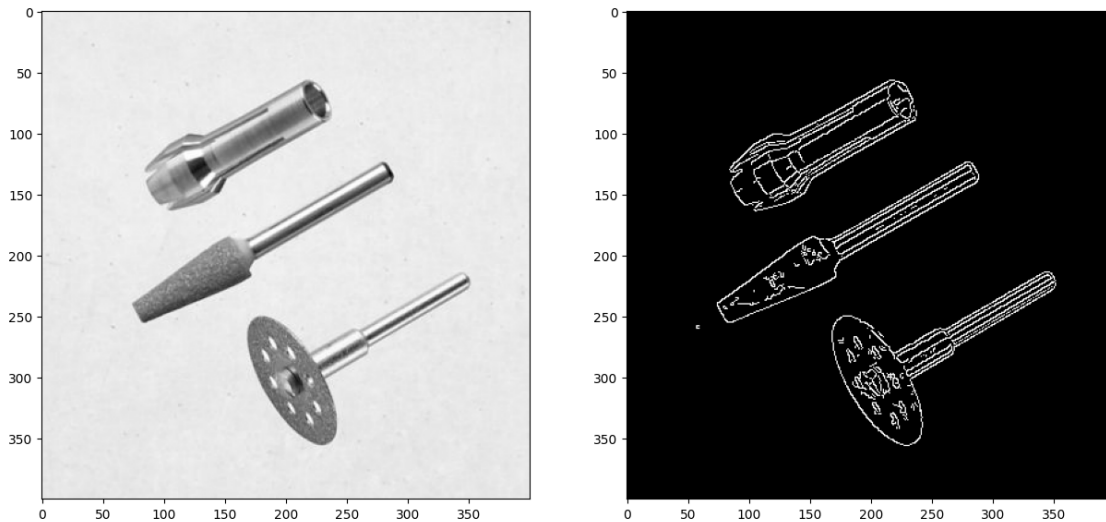
canny_channels = [cv2.Canny(channel, 100, 200) for channel in channels]

canny_combined = cv2.merge(canny_channels)

plt.figure(figsize=(15,10))
plt.subplot(1, 2, 1)
plt.imshow(img1, cmap='gray')

```

```
plt.subplot(1, 2, 2)
plt.imshow(canny_combined, cmap='gray')
plt.show()
```



1.4 Ejercicio 4: Detección de Contornos con Umbralizado

Objetivo: Detectar y dibujar contornos en una imagen a color utilizando umbralización y `cv2.findContours()`.

Pasos: 1. Convierte la imagen a escala de grises usando `cv2.cvtColor()`. 2. Genera un umbralizado en la imagen utiliza identifica cual es el mejor para la imagen. 3. Detectar contornos con `cv2.findContours()`. 4. Dibujar contornos sobre la imagen original con `cv2.drawContours()` 4. Muestra el resultado obtenido con `plt.imshow()`.

```
[33]: image = cv2.imread('/content/drive/MyDrive/Colab Notebooks/
↳2_3_Segmentacion_de_Imagenes/data/herramientas.jpg')

gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

_, binary = cv2.threshold(gray, 127, 255, cv2.THRESH_BINARY)
contours, _ = cv2.findContours(binary, cv2.RETR_EXTERNAL, cv2.
↳CHAIN_APPROX_SIMPLE)
binary_contours = image.copy()
cv2.drawContours(binary_contours, contours, -1, (0, 255, 0), 2)

_, binary_inv = cv2.threshold(gray, 127, 255, cv2.THRESH_BINARY_INV)
contours, _ = cv2.findContours(binary_inv, cv2.RETR_EXTERNAL, cv2.
↳CHAIN_APPROX_SIMPLE)
binary_inv_contours = image.copy()
cv2.drawContours(binary_inv_contours, contours, -1, (0, 255, 0), 2)
```

```

binary_adaptive = cv2.adaptiveThreshold(gray, 255, cv2.
    ↳ADAPTIVE_THRESH_GAUSSIAN_C, cv2.THRESH_BINARY_INV, 11, 2)
contours, _ = cv2.findContours(binary_adaptive, cv2.RETR_EXTERNAL, cv2.
    ↳CHAIN_APPROX_SIMPLE)
adaptive_contours = image.copy()
cv2.drawContours(adaptive_contours, contours, -1, (0, 255, 0), 2)

edges = cv2.Canny(gray, 50, 150)
contours, _ = cv2.findContours(edges, cv2.RETR_EXTERNAL, cv2.
    ↳CHAIN_APPROX_SIMPLE)
adaptive_canny_contours = image.copy()
cv2.drawContours(adaptive_canny_contours, contours, -1, (0, 255, 0), 2)

plt.figure(figsize=(12, 4))

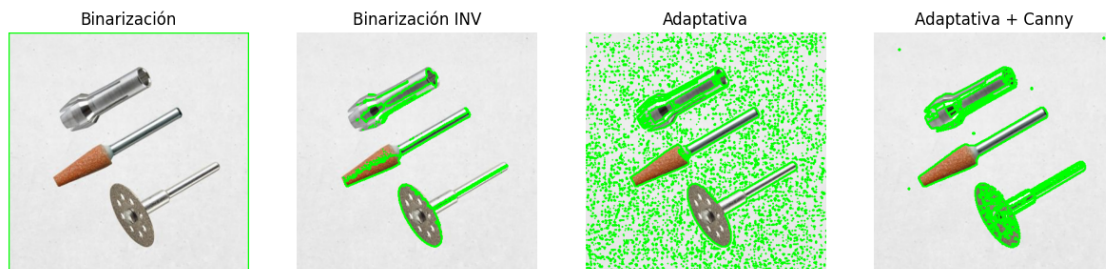
titles = ["Binarización", "Binarización INV", "Adaptativa", "Adaptativa + Canny"]
images = [binary_contours, binary_inv_contours, adaptive_contours,
    ↳adaptive_canny_contours]

plt.figure(figsize=(15, 10))
for i in range(4):
    plt.subplot(1, 4, i + 1)
    plt.imshow(cv2.cvtColor(images[i], cv2.COLOR_BGR2RGB))
    plt.title(titles[i])
    plt.axis("off")

plt.show()

```

<Figure size 1200x400 with 0 Axes>



1.5 Ejercicio 5: Segmentación de Regiones

Objetivo: Segmentar objetos en imágenes a color utilizando umbralización en el espacio de color HSV.

Pasos: 1. Cargar la imagen martillo con `cv2.imread()`. 2. Convierte la imagen al modelo HSV con `cv2.cvtColor()`. 3. Definir un rango de color amarillo en HSV 3. Crear una máscara binaria con `cv2.inRange()`. 4. Aplicar la máscara a la imagen original para extraer solo el color amarillo utiliza `cv2.bitwise_and()`. 5. Visualizar la imagen original y la imagen con la region detectada `plt.subplot()`.

```
[41]: image_path = '/content/drive/MyDrive/Colab Notebooks/
↳2_3_Segmentacion_de_Imagenes/data/martillo.jpg'
image = cv2.imread(image_path)

image_hsv = cv2.cvtColor(image, cv2.COLOR_BGR2HSV)

lower_yellow = np.array([20, 100, 100])
upper_yellow = np.array([35, 255, 255])

mask_yellow = cv2.inRange(image_hsv, lower_yellow, upper_yellow)

segmented_image = cv2.bitwise_and(image, image, mask=mask_yellow)

image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
segmented_rgb = cv2.cvtColor(segmented_image, cv2.COLOR_BGR2RGB)

plt.figure(figsize=(12, 4))
plt.subplot(1,2,1)
plt.imshow(image_rgb)
plt.title('Imagen Original')
plt.axis('off')
plt.subplot(1,2,2)
plt.imshow(segmented_rgb)
plt.title('Imagen Segmentada')

plt.show()
```

Imagen Original



Imagen Segmentada

